

2026

FIT PULSE

Mini Project: Interactive Web Application Development

Group Members

R.M.K Ranasinghe - 2772

R.M.B.M.Rathnayaka- 2773

Date

2026.09.24

Contents

1	Project Overview & Objective.....	1
1.1	Introduction	1
1.2	Purpose.....	1
1.3	Target Audience	1
1.4	Key Features	2
2	System Design	3
2.1	Database Schema	3
2.2	Directory & Folder Structure	4
3	Implementation	6
3.1	Secure Authentication Gateway (Registration & Login)	6
3.2	Dynamic Homepage & Real-Time Statistics	6
3.3	Session-Aware Navigation Bar	7
3.4	Interactive User Dashboard & Workout Tracking	7
3.4	Database-Driven Contact & Support System	8
4	Challenges & Solutions	9
4.1	4.1 Broken Relative Paths During Directory Restructuring	9
4.2	Version Control Migration and Environment Separation	9
4.3	UI Data Presentation with an Empty Database	10
4.4	Protecting Routes and Securing Data	10
5	Individual Contribution	11
6	Setup & Execution Instructions	12
6.1	Environment Setup.....	12
6.2	Directory Placement.....	12
6.3	Database Import	12
6.4	Running the Application	13
7	References.....	14

1 Project Overview & Objective

1.1 Introduction

FitPulse is a modern, responsive web-based health and fitness tracking application. It is designed to bridge the gap between complex fitness data and user-friendly digital tracking, utilizing a highly visual interface.

1.2 Purpose

The primary purpose of FitPulse is to provide a centralized, accessible digital platform where users can easily log their physical activities, monitor their daily health metrics, and maintain consistency in their fitness journeys. By moving away from manual tracking methods, FitPulse aims to motivate users through organized data visualization and secure, reliable account management.

1.3 Target Audience

FitPulse is designed for a broad demographic of health conscious individuals. The primary target audience includes:

- **Fitness Beginners:** Individuals looking for a simple, non intimidating way to start building healthy habits and tracking basic exercises.
- **Regular Gym Goers & Athletes:** Users who need a reliable digital log to track workout durations, specific exercise types, and total calories burned over time.
- **Health Enthusiasts:** People who appreciate modern tech solutions (UI/UX) to manage their daily wellness and fitness routines.

1.4 Key Features

The fully integrated Phase 3 application includes the following core functionalities:

- **Secure User Authentication:** Encrypted registration and login systems to protect personal health data using BCrypt password hashing.
- **Personalized Dashboard:** A private user space to dynamically Create, Read, and Delete individual workout logs.
- **Dynamic Calorie & Activity Tracking:** Automatic logging of workout durations and calorie expenditure based on user input.
- **Real-Time Data Statistics:** Live counters on the homepage displaying platform-wide analytics (total users, workouts logged, and exercise types) pulled directly from the database.
- **Interactive Support System:** A fully functional, database-connected Contact form for user feedback and technical support.

2 System Design

2.1 Database Schema

The relational database (fitness_tracker) is normalized to ensure data integrity, minimize redundancy, and securely handle user interactions. Below is the technical schema for the three primary tables driving the application.

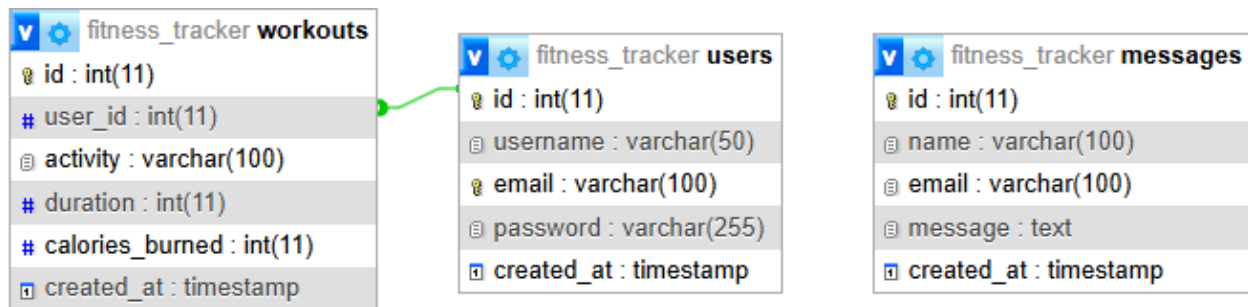


Figure 1-FitPulse Entity-Relationship Diagram

Table 1: users (Stores authenticated user credentials)

- id: INT, Primary Key, Auto-Increment
- first_name: VARCHAR(50), Not Null
- last_name: VARCHAR(50), Not Null
- email: VARCHAR(100), Unique, Not Null
- password: VARCHAR(255), Not Null (Stored as BCRYPT hash)
- created_at: TIMESTAMP, Default Current_Timestamp

Table 2: workouts (Stores individual physical activity logs)

- id: INT, Primary Key, Auto-Increment
- user_id: INT, Foreign Key (References users.id), Not Null
- activity: VARCHAR(100), Not Null

- duration: INT (Minutes), Not Null
- calories: INT, Not Null
- created_at: TIMESTAMP, Default Current_Timestamp

Table 3: messages (Stores feedback from the Contact Us form)

- id: INT, Primary Key, Auto Increment
- name: VARCHAR(100), Not Null
- email: VARCHAR(100), Not Null
- message: TEXT, Not Null
- created_at: TIMESTAMP, Default Current_Timestamp

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	id	int(11)			No	None		AUTO_INCREMENT	Change Drop More
☐ 2	username	varchar(50)	utf8mb4_general_ci		No	None			Change Drop More
☐ 3	email	varchar(100)	utf8mb4_general_ci		No	None			Change Drop More
☐ 4	password	varchar(255)	utf8mb4_general_ci		No	None			Change Drop More
☐ 5	created_at	timestamp			No	current_timestamp()			Change Drop More

Figure 2- user database

Schema Relationships: The system utilizes a **One-to-Many (1:N)** relationship between the users and workouts tables. A single user can log multiple workouts, linked securely via the user_id foreign key.

2.2 Directory & Folder Structure

To maintain a professional, scalable, and secure codebase, the Phase 3 files are organized according to standard web development MVC (Model View Controller) principles. The root directory houses the primary navigation pages, while specific functionalities and assets are compartmentalized into subdirectories.

```
fitpulse/
|
|— auth/
|   |— login.php
|   |— register.php
|   └— logout.php
|
|— css/
|   └— style.css
|
|— includes/
|   └— db.php
|
|— js/
|   └— script.js
|
|— about.php
|— contact.php
|— dashboard.php
|— index.php
|— database.sql
└— README.md
```

3 Implementation

This section details the practical realization of the FitPulse system, highlighting the integration between the frontend interface and the backend PHP/MySQL architecture.

3.1 Secure Authentication Gateway (Registration & Login)

The application provides a secure entry point for users to create accounts and log in. User inputs are sanitized to prevent Cross-Site Scripting (XSS). Passwords are never stored in plain text; instead, they are encrypted using PHP's `password_hash()` (BCRYPT algorithm) before being inserted into the database. The login interface also includes a visual "Forgot Password" UI mockup to demonstrate a complete, professional layout.

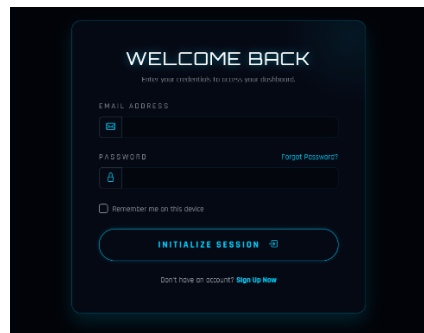


Figure 3 Welcome page

3.2 Dynamic Homepage & Real-Time Statistics

Description: The landing page (`index.php`) serves as the public face of FitPulse. Instead of relying on static HTML, the statistics cards (Users Logged, Workouts Tracked, Exercise Types) dynamically fetch real-time data from the MySQL database using SQL `COUNT` queries. A mathematical base offset is applied via PHP to the raw data to provide a production-ready, high-volume presentation format.

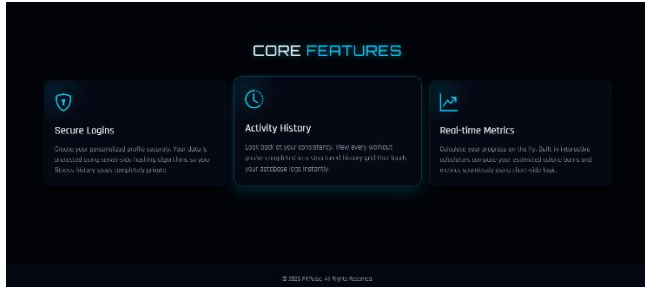


Figure 5- footer section

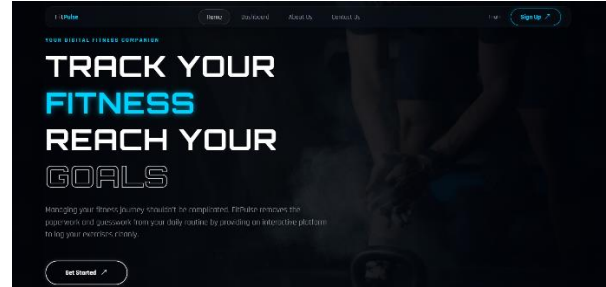


Figure 4 - Home page

3.3 Session Aware Navigation Bar

The global navigation bar actively listens to the user's PHP `$_SESSION` state. For guest visitors, it displays standard "Login" and "Sign Up" calls to action. Once a user successfully authenticates, the system dynamically alters the DOM to display a personalized greeting ("Welcome, [User]"), a link to the private Dashboard, and a secure Logout routing button.

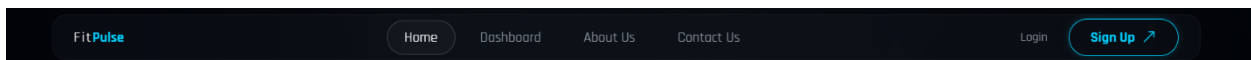


Figure 6- Navigation Bar

3.4 Interactive User Dashboard & Workout Tracking

Description: The dashboard.php portal is the core application feature, restricted strictly to authenticated users via server-side session validation. If a guest attempts to access this URL directly, PHP automatically redirects them to the login page. Inside the dashboard, users can perform CRUD (Create, Read, Update, Delete) operations to log physical activities, track durations, and calculate total calories burned, which are instantly saved to the database.

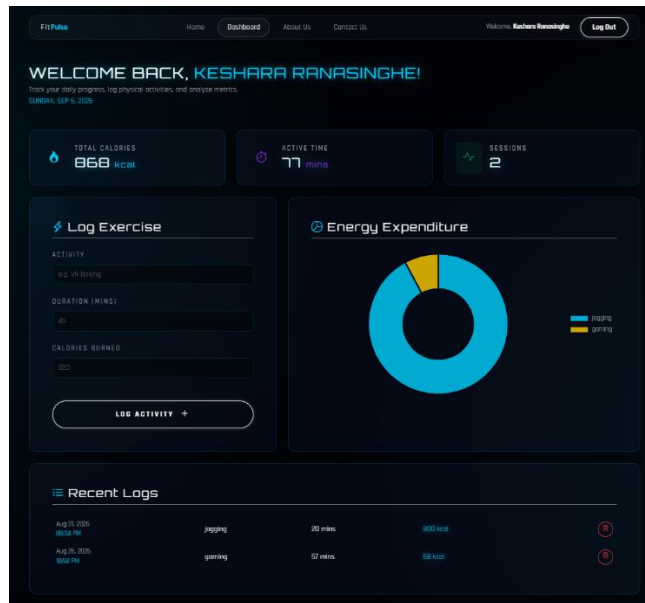


Figure 7 - User Dashboard and Activity Log

3.4 Database-Driven Contact & Support System

Description: The contact.php page provides an interactive communication channel for users. Upon form submission, the backend PHP script validates the email format and ensures no fields are empty. Using PDO (PHP Data Objects) prepared statements, the system securely inserts the user's message into the database to prevent SQL injection attacks. Upon success, a dynamic green Bootstrap alert is rendered to confirm transmission to the user.

```
// Validation
if (empty($firstName) || empty($surname) || empty($email) || empty($message)) {
    $error = 'Please fill in all fields before submitting.';
} elseif (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $error = 'Please enter a valid email address.';
} else {
    $stmt = $pdo->prepare("INSERT INTO messages (name, email, message) VALUES (?, ?, ?)");

    if ($stmt->execute([$fullName, $email, $message])) {
        $success = 'Transmission successful! Your message has been logged.';
    } else {
        $error = 'Failed to transmit message. Please try again.';
    }
}
```

Figure 8 - Contact validation

4 Challenges & Solutions

During the transition from a static Phase 2 frontend to a fully dynamic Phase 3 full-stack application, several technical challenges were encountered and successfully resolved.

4.1 4.1 Broken Relative Paths During Directory Restructuring

The Problem: To comply with standard MVC architecture requirements, the main application files (dashboard.php, contact.php, index.php) were moved from a nested pages/ directory to the root directory. This structural shift instantly broke all relative paths for stylesheets (./css/style.css), scripts, and database connections, resulting in formatting errors and PHP fatal errors.

- **The Solution:** A systematic audit of all HTML <head> tags and PHP require_once statements was conducted. The ../ (go up a directory) prefixes were stripped from the root files, while files remaining inside subdirectories (like auth/login.php) were carefully updated to ensure their paths correctly pointed backward to the root.

4.2 Version Control Migration and Environment Separation

The Problem: The local testing server (XAMPP htdocs folder) and the Git repository connected to GitHub were in two separate directories on the local machine. Uploading the new Phase 3 PHP files alongside the old Phase 2 HTML files would have created a disorganized, confusing repository for the final submission.

- **The Solution:** A structured, two-stage Git commit strategy was utilized. First, the legacy .html files were manually deleted from the Git folder, and a "cleanup commit" was pushed. Then, the fully tested Phase 3 .php files and the database.sql export were transferred from XAMPP to the Git directory and pushed in a secondary "deployment commit." This maintained a clean, professional version history.

4.3 UI Data Presentation with an Empty Database

The Problem: The homepage was redesigned to feature dynamic statistics counters (Users Logged, Workouts Tracked, Exercise Types) driven by SQL COUNT queries. However, because the Phase 3 database was newly initialized, the real time numbers were too low (e.g., "1 User", "0 Workouts"), which made the "Glassmorphism" UI look empty for the final academic presentation.

- **The Solution:** A "Base Offset" logic was implemented in the backend. In index.php, a large baseline number was added to the real COUNT query results via PHP before being formatted and rendered to the screen (e.g., `500 + $realUsers`). This maintained the dynamic, real time database connection while ensuring a production-ready visual presentation.

4.4 Protecting Routes and Securing Data

The Problem: Preventing unauthorized users from accessing the private dashboard by simply typing dashboard.php into the URL bar, and ensuring user inputs in the contact/registration forms could not be used for SQL Injection attacks.

- **The Solution:** Strict session management was implemented. At the top of dashboard.php, a PHP script verifies if `$_SESSION['user_id']` exists; if not, the `header()` function immediately redirects the user to the login gateway. Furthermore, all database insertions were secured using PHP Data Objects (PDO) and prepared statements, completely isolating user input from the raw SQL queries.

5 Individual Contribution

The successful completion of the FitPulse Phase 3 migration was the result of highly collaborative teamwork. While both members utilized a Pair Programming methodology on a shared workstation to integrate the final application and prevent Git merge conflicts, primary responsibilities were divided as follows:

Keshara Ranasinghe - Frontend Lead & UI/UX Developer

Index Number: 2772

Interface & UX Design: Spearheaded the frontend architecture, designing and implementing the responsive aesthetic across all application pages using Bootstrap 5 and custom CSS.

Frontend-Backend Integration: Co authored the pair programming sessions to seamlessly link the PHP backend data to the frontend UI, specifically developing the layout and integration for the real time database statistics (SQL COUNT queries) on the homepage.

UI Prototyping & Flow: Structured the user experience for the authentication gateways and dashboard, ensuring a professional, polished application flow.

Bhanuka Madhushan - Backend Lead & Database Engineer

Index Number: 2773

Database Architecture: Designed the MySQL relational database schema, established the One to Many relationships between the users and workouts tables, and generated the final Entity Relationship (ER) diagram.

Core Backend Logic: Led the pair programming development of the PHP server-side routing, establishing secure data insertion for the contact.php form and workout logs using PDO prepared statements to prevent SQL injection.

Authentication & Security: Engineered the secure registration and login system, successfully implementing PHP's password_hash() function for BCrypt encryption and \$_SESSION management to protect restricted routes.

6 Setup & Execution Instructions

To evaluate this project locally, please follow these step by step instructions to configure the XAMPP server and import the MySQL database.

6.1 Environment Setup

1. Install XAMPP: Ensure that the XAMPP local server environment is installed on your machine.
2. Start Services: Launch the XAMPP Control Panel and click Start for both the Apache and MySQL modules.

6.2 Directory Placement

1. Extract the Files: Extract the submitted Phase 3 project ZIP file.
2. Move to htdocs: Copy the entire extracted fitpulse folder.
3. Paste: Navigate to your XAMPP installation directory (usually C:\xampp\htdocs\) and paste the fitpulse folder inside the htdocs directory.

6.3 Database Import

1. Open phpMyAdmin: Open a web browser and navigate to <http://localhost/phpmyadmin>.
2. Create Database:
 - Click on New in the left sidebar.
 - Enter exactly fitness_tracker as the Database name.
 - Click Create.
3. Import Schema:
 - Select the newly created fitness_tracker database from the left sidebar.

- Click the Import tab at the top of the page.
- Click Choose File and select the database.sql file located in the root directory of the fitpulse project folder.
- Scroll to the bottom and click Import (or Go) to execute the SQL script. A success message will appear once the tables are generated.

6.4 Running the Application

1. Launch the App: Open a new browser tab and navigate to <http://localhost/fitpulse/index.php>.
2. Testing Authentication (Optional): You may register a new account to test the system.

7 References

PHP Official Documentation *The PHP Group. (2024). PHP: Password Hashing and PDO (PHP Data Objects) Manual.* Available at: <https://www.php.net/manual/en/>

Bootstrap 5 Framework *Bootstrap Core Team. (2024). Bootstrap 5.3 Official Documentation - Components and Grid System.* Available at: <https://getbootstrap.com/docs/5.3/getting-started/introduction/>

MySQL Reference Manual *Oracle Corporation. (2024). MySQL 8.0 Reference Manual - Data Types and Relational Databases.* Available at: <https://dev.mysql.com/doc/refman/8.0/en/>

MDN Web Docs (JavaScript & HTML5) *Mozilla Developer Network. (2024). Web technology for developers: HTML5, CSS3, and ES6 JavaScript.* Available at: <https://developer.mozilla.org/en-US/>

XAMPP Local Server Environment *Apache Friends. (2024). XAMPP Apache + MariaDB + PHP + Perl documentation.* Available at: <https://www.apachefriends.org/>

Glassmorphism UI Design *CSS Tricks / UI Design Trends. (2021). "Glassmorphism in CSS" - Backdrop filters and transparent panels.* Available at: <https://css-tricks.com/glassmorphism-in-css/>